ARCHITECTURE DESIGN

Refactoring Racing to Reinforcement Learning

A concrete design for selectable PPO and SAC training with RLtools, CUDA acceleration, headless simulation, deterministic evaluation, and Win32 visualization.

Scope	Baseline	Target
Project	C++20 / Win32 / Direct2D	C++20 + RLtools + optional CUDA
Current trainer	64-genome genetic algorithm	PPO and SAC selectable at runtime
Control	3 high-level continuous targets	Tanh-Gaussian policy, same action contract
Simulation	Single UI env + GA worker copies	Headless batched environments + viewer

Prepared from direct inspection of racing/code/racing.h, simulation.cpp, training.cpp, main.cpp, and rendering.cpp.

Design date: 04 September 2026 | Workspace: C:/Users/Massimo/Documents/Playground/racing

Contents

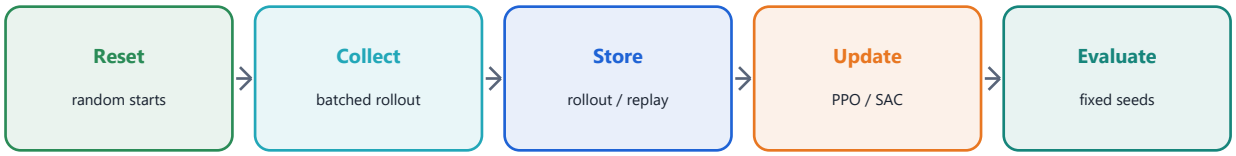
1. Executive decision

Recommended direction

Keep the existing high-level action contract and simulation physics for the first RL milestone. Extract them into a platform-neutral headless core, wrap that core as an RLtools environment, and implement two trainer adapters: PPO for the first correctness baseline and SAC for the sample-efficient alternative. Train neural networks on CUDA, but keep the current scalar physics on CPU until profiling proves that a GPU simulation rewrite is worthwhile.

The existing project is closer to an RL-ready environment than its genetic trainer suggests. It already exposes deterministic reset, observation, action, reward, termination, and fixed-step episode evaluation. The refactor should preserve these validated mechanics while removing GA-specific types from the application state and moving training outside the render loop.

Decision	Choice	Reason
Primary library	RLtools, pinned revision	Native C++, PPO and SAC support, CUDA backends, and an environment abstraction suited to this codebase.
First algorithm	PPO	Simpler rollout semantics and easier parity testing against complete fixed-start episodes.
Second algorithm	SAC	Replay-based learning can reuse transitions and should improve sample efficiency after the environment contract is stable.
Action space	3-D normalized continuous	Maps cleanly to desired speed, lateral offset, and heading offset without rewriting the low-level controller.
Simulation device	CPU first	The current environment is branch-heavy and small; network updates are the clearest initial CUDA workload.
Build system	CMake as source of truth	Generates Visual Studio projects while handling RLtools and optional CUDA more reliably than hand-editing vcxproj files.



2. Current system assessment

2.1 What can be retained

Current component	Evidence	Refactor disposition
Environment state	RacingEnv owns track/config pointers, controller state, history, RNG, and CarState (racing.h).	Retain layout initially; move Windows-independent declarations into env/*.h.
Observation	racing_env_observe builds a 64-float vector from dynamics, 5 history samples, 7 lidar rays, and 4 lookahead points (simulation.cpp:295-393).	Retain values for parity; add schema/version and running normalization outside the environment.
Action	NeuralPolicy emits desired speed, lateral offset, and heading offset; the deterministic vehicle controller converts them to pedals and steering (simulation.cpp:396-438, 1268-1383).	Retain controller-mediated action mode for v1; normalize policy actions to [-1, 1].
Reward	Progress, time, steering-change, lateral-demand, off-track, stationary, and lap bonuses are already separated in RacingStepResult (simulation.cpp:1525-1587).	Keep decomposed terms; make weights configurable and log every term.
Episode lifecycle	Reset-at-position and fixed-step evaluation already exist; three starts are used by GA (training.cpp:52-100, 121-186).	Generalize start randomization; distinguish terminated from truncated.
Rendering	Direct2D only reads environment and training telemetry (rendering.cpp).	Keep as a consumer; remove any ownership of trainer internals.

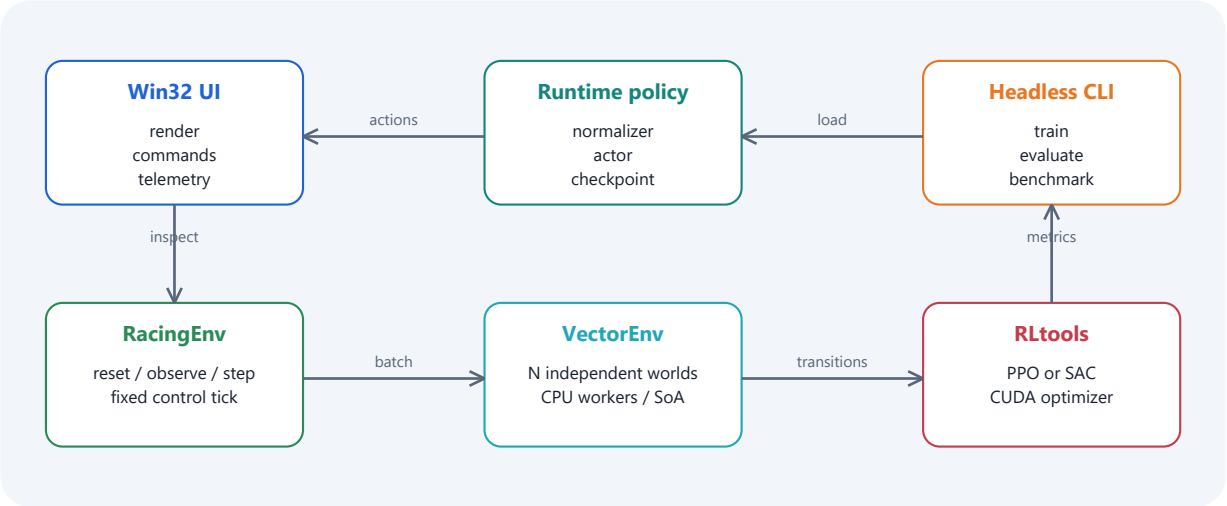
2.2 Coupling that must be removed

- racing.h is an umbrella header. It mixes Windows, Direct2D, simulation, neural-network, GA, rendering, and application types. This prevents a clean headless training target.
- TrainingContext is GA-shaped. Population arrays, genome conversion, Windows thread-pool work items, window messages, and the active policy pointer are part of one structure.
- The policy representation is fixed in the environment layer. NeuralPolicy and NN_GENOME_COUNT hard-code a 64-32-16-3 network and genetic serialization.
- Training is UI-controlled. WM_GA_GENERATION_COMPLETE and button semantics couple progress to the Win32 message pump.
- Run mode uses variable wall-clock elapsed. Training uses exactly 1/60 s while the viewer passes measured frame time. Evaluation should use a fixed control tick to remain deterministic.
- Checkpoint format is GA-specific. RACENN4 stores only flat genes and cannot preserve optimizer state, observation statistics, algorithm settings, or schema versions.

2.3 Baseline dimensions and semantics

Item	Current value	RL interpretation
Observation size	64 floats	6 instantaneous + 35 history + 7 lidar + 16 lookahead features.
Action size	3 floats	Desired speed, lateral offset, and heading offset.
Policy network	64 -> 32 -> 16 -> 3, tanh hidden	Deterministic GA policy; replace with stochastic actor and value/Q networks.
Physics substep	max 0.01 s	Environment integrates multiple substeps inside each control step.
Training control step	1/60 s	Use this for parity; later benchmark action repeat 2 for 30 Hz decisions.
Episode cap	12,000 steps = 200 s	Must become a truncation, not a terminal failure.
Parallelism	64 Windows worker contexts	Reusable idea, but batch environments independently of algorithm.

3. Target architecture



The target separates four concerns: deterministic racing dynamics, batched environment execution, algorithm-specific learning, and presentation. Both algorithms consume the same observation/action/reward contract. The Win32 program becomes an evaluator and live telemetry client; it does not perform gradient updates on the UI thread.

3.1 Proposed source tree

The following layout is intentionally incremental: existing algorithms can be moved with minimal renaming before deeper cleanup.

```
racing/
CMakeLists.txt
cmake/Dependencies.cmake
code/
core/math.h, track.h, track.cpp
env/racing_env.h, racing_env.cpp, observation.h, reward.h
control/vehicle_controller.h, vehicle_controller.cpp
rl/rl_config.h, policy_runtime.h, checkpoint.h
rl/rltools_env_adapter.h
rl/ppo_trainer.h, ppo_trainer.cpp
rl/sac_trainer.h, sac_trainer.cpp
app/main_win32.cpp, rendering.cpp
tools/train_main.cpp, evaluate_main.cpp, benchmark_main.cpp
config/ppo_baseline.json, sac_baseline.json, reward_v1.json
data/checkpoints/, data/runs/
tests/
```

3.2 Dependency rules

Layer	May depend on	Must not depend on
core + env	C++ standard library or restricted POD subset	Windows, Direct2D, RLtools, CUDA, UI state
control	core + env configuration	Renderer, trainer, replay/rollout storage
rl adapter	core + env + RLtools	Win32 window handles and Direct2D
train tools	env + rl + logging	Renderer or message pump
app	env + policy runtime + renderer	Optimizer state or algorithm loop internals

3.3 Runtime processes

- racing_train.exe: headless, fixed-step, seeded, batched environment execution. Selects `--algo ppo` or `--algo sac`.
- racing_evaluate.exe: loads a frozen checkpoint and runs an immutable evaluation suite across fixed seeds and starts.
- racing.exe: retains the current Direct2D viewer, loads inference-only policy snapshots, and optionally tails a metrics file produced by training.
- racing_benchmark.exe: measures environment steps/s, inference batches/s, optimizer updates/s, and transfer overhead for CPU and CUDA modes.

4. Environment contract

4.1 Algorithm-neutral C++ API

```
struct EnvConfig {
float physics_max_step = 0.01f;
float control_step = 1.0f / 60.0f;
int action_repeat = 1;
int max_episode_steps = 12000;
StartMode start_mode = StartMode::Curriculum;
};

struct StepResult {
Observation next_observation;
float reward;
bool terminated; // failure or completed task
bool truncated; // time limit or external cutoff
RewardTerms terms;
EpisodeInfo info;
};

class RacingEnv {
public:
Observation reset(uint64_t seed, const ResetOptions&);
StepResult step(const NormalizedAction&);
const CarState& state() const;
};
```

Critical bootstrap rule

When terminated is true, the value target must not bootstrap from the next state. When only truncated is true, PPO GAE and SAC targets should bootstrap from the final observation. The current 12,000-step cap therefore becomes a truncation.

4.2 Observation schema

Version obs-v1 keeps the 64 existing features to make regression tests possible. A schema object owns indices and scaling constants; no trainer should rely on numeric offsets copied from macros. The policy-facing vector is transformed from current [0, 1] encodings into approximately centered values and then normalized using running mean and variance.

Group	Count	Current encoding	Recommended treatment
Instantaneous dynamics	6	Longitudinal/lateral velocity, steering, yaw, longitudinal/lateral acceleration	Center signed channels; clamp normalized output to [-10, 10].
History	35	5 samples x 7 values at 0.05 s intervals	Keep for v1; later compare frame stacking against an RNN only after a stable baseline.
Lidar	7	Distance / range	Keep [0, 1], optionally map no-hit 1.0 consistently.
Lookahead	16	Lateral, forward, angle, curvature for 4 points	Center signed channels and preserve speed-dependent lookahead distances.

Observation-normalization statistics belong to the trainer/checkpoint, not RacingEnv. Evaluation freezes the statistics. Include NaN/Inf assertions before writing a transition and record the first offending state.

4.3 Continuous action schema

Both PPO and SAC use a tanh-squashed Gaussian policy with a normalized action a in [-1, 1]^3. The environment maps it deterministically to the current controller targets:

Policy component	Mapping	Physical range
a[0] speed	desired_speed = (a[0] + 1) / 2 * max_speed	0 to 91.67 m/s at 330 km/h
a[1] lateral	desired_lateral_offset = a[1] * max_lateral_offset	-5.5 to +5.5 m
a[2] heading	desired_heading_offset = a[2] * max_heading_offset	-20 to +20 degrees

PPO stores the pre-squash Gaussian log probability with the tanh Jacobian correction. SAC uses the same correction in its entropy objective. Deterministic evaluation uses tanh(mean), not a random sample.

4.4 Alternative low-level action mode

A later direct-controls mode may expose steering, throttle, and brake. Do not make it the first milestone: it increases exploration difficulty and invalidates comparisons with the current controller. If added, prefer a two-dimensional action (steering, longitudinal command) where positive longitudinal means throttle and negative means brake, preventing simultaneous pedals by construction.

4.5 Reset and domain randomization

Phase	Start distribution	Randomization
Parity	Existing start/finish + sector 1 + sector 2	None; fixed seed suite.
Coverage	Uniform centerline position with small lateral/heading offsets	Initial speed 0-40 m/s.
Robustness	Full track, including corner entries	Mass, grip/lateral limit, power, sensor noise +/-5 percent.
Generalization	Hold-out centerline segments or track files	Wider parameter ranges, but never during final evaluation.

5. Reward design

5.1 Preserve decomposition, revise scaling

The current reward is structurally sound because each term is separately observable. The main RL risk is scale imbalance: per-step normalized progress is small, while lap and failure bonuses are discontinuous. Start with a parity profile and introduce one change at a time, recording every component.

```
r_t = w_p * delta_s / track_length
- w_time * dt
- w_steer * abs(delta steering_command)
- w_lat * excess_lateral_ratio * dt
- w_conflict * throttle * brake * dt
+ terminal_bonus
```

Term	Current default	Baseline RL setting	Rationale
Progress per lap	+10	+10	Dense, direction-sensitive task signal.
Time	-0.001 / s	-0.01 / s initially	Discourage dithering without dominating progress.
Steering change	-0.002 * abs(delta)	Retain	Promotes smooth high-level targets.
Excess lateral	-0.01 / s scaled	Retain	Penalizes physically infeasible demand.
Off track	-1	Start -2, tune after curves	Needs enough weight to beat reward hacking near boundaries.
Stationary	-1	-1	Prevents no-op local optimum.
Lap completion	+5 + up to 5 speed	Use +2 initially; log speed separately	Reduces sparse bonus dominance while retaining task completion.

5.2 Reward invariants

- Forward progress around the start-line wrap must remain continuous; backward crossings must produce negative progress.
- A transition must receive exactly one terminal outcome. Off-track takes precedence over stationary and lap completion, matching current behavior.
- Reward totals must equal the sum of serialized component terms within floating-point tolerance.
- Changing control frequency must not silently rescale time-based terms; every per-second penalty multiplies by dt.
- Evaluation reports lap time, completion rate, off-track rate, and smoothness independently of shaped return.

5.3 Curriculum gates

Stage	Objective	Promotion gate
A - move	Non-stationary forward progress from fixed starts	>95% non-stationary over 100 evaluation episodes
B - remain	Track coverage without leaving	>80% reach next sector from each fixed start
C - finish	Complete full laps	>60% lap completion across 10 fixed seeds
D - race	Improve lap time and smoothness	Median lap improves without degrading 10th percentile completion

6. PPO design

PPO is the recommended first implementation because its on-policy rollout can be checked step-by-step against the existing evaluator. It requires an actor and a scalar state-value function, generalized advantage estimation (GAE), clipped policy updates, and minibatch epochs.

6.1 Networks

Component	Input	Proposed topology	Output
Actor	64 normalized observations	128 tanh -> 128 tanh	3 means + 3 trainable log standard deviations
Critic	64 normalized observations	128 tanh -> 128 tanh	1 value estimate

Use separate actor and critic trunks for clearer diagnostics. A shared trunk is smaller but allows the value loss to alter policy features. Orthogonal initialization with small actor-output gain is a sensible baseline if supported cleanly by the chosen RLtools configuration.

6.2 Rollout and loss

```
delta_t = r_t + gamma * bootstrap_mask * V(s_{t+1}) - V(s_t)
A_t = delta_t + gamma * lambda * continuation_mask * A_{t+1}
ratio_t = exp(log pi_new(a_t|s_t) - log pi_old(a_t|s_t))
L_policy = -mean(min(ratio*A, clip(ratio, 1-eps, 1+eps)*A))
L_total = L_policy + c_v * L_value - c_e * entropy
```

Normalize advantages across the complete rollout batch. Preserve final observations for truncations. Shuffle transition indices each epoch, clip gradient norm, and stop early if approximate KL exceeds the configured threshold.

6.3 PPO baseline configuration

Parameter	Initial value	Notes
Parallel environments	128	CPU headless instances; benchmark 64, 128, 256.
Rollout horizon	256 decisions	32,768 transitions per update at 128 envs.
Control frequency	60 Hz	Parity first; action-repeat 2 experiment later.
Discount gamma	0.995	Long racing horizon; compare 0.99 only after baseline.
GAE lambda	0.95	Bias/variance baseline.
Clip epsilon	0.20	Log clip fraction and approximate KL.
Learning rate	3e-4	Linear decay to 3e-5 is optional after stability.
Epochs / minibatch	10 / 2048	16 minibatches per epoch for 32,768 samples.
Value coefficient	0.5	Enable value clipping if available.
Entropy coefficient	0.003	Track action std; decay only if exploration remains excessive.
Max gradient norm	0.5	Record unclipped and clipped norms.
Target KL	0.02	Early-stop epochs when exceeded.

6.4 PPO update pseudocode

```
for update in range(total_updates):
rollout.clear()
for t in range(horizon):
action, logp, value = actor_critic.sample(obs_batch)
next_obs, reward, terminated, truncated = vector_env.step(action)
rollout.add(obs, action, reward, value, logp, flags, final_obs)
obs_batch = reset_finished(next_obs)
rollout.compute_gae(last_values, gamma, lambda)
for epoch in shuffled_epochs:
optimize_clipped_policy_value_entropy(minibatches)
if approximate_kl > target_kl: break
periodic_evaluate_and_checkpoint()
```

7. SAC design

SAC is the second selectable algorithm. It is off-policy: transitions enter a replay buffer and may be used for many gradient updates. A stochastic actor is trained against the minimum of two critics while entropy temperature alpha is tuned automatically.

7.1 Networks and targets

Component	Input	Topology	Output
Actor	64 observations	256 ReLU -> 256 ReLU	3 means + 3 bounded log standard deviations
Critic Q1	64 observations + 3 actions	256 ReLU -> 256 ReLU	Scalar Q
Critic Q2	64 observations + 3 actions	256 ReLU -> 256 ReLU	Scalar Q
Target critics	Same as critics	Polyak copies	Bootstrapped target only

Clamp actor log standard deviation to a safe range such as [-20, 2] before sampling. Use the reparameterization trick and the tanh Jacobian correction. The replay record must contain both termination flags and the true final observation for time-limit truncations.

7.2 SAC objectives

```
y = r + gamma * bootstrap_mask * (min(Q1_target, Q2_target) - alpha * log pi(a'|s'))
L_Q = mean((Q1 - y)^2 + (Q2 - y)^2)
L_actor = mean(alpha * log pi(a|s) - min(Q1, Q2))
L_alpha = -mean(log_alpha * (log pi(a|s) + target_entropy))
```

7.3 SAC baseline configuration

Parameter	Initial value	Notes
Parallel environments	64	Replay generation scales independently from update batches.
Replay capacity	1,000,000	Approx. 300-400 MB depending on packed record layout.
Warm-up	20,000 transitions	Uniform random normalized actions.
Batch size	1024	Large enough for efficient CUDA matmuls.
Discount gamma	0.995	Match PPO for comparison.
Target update tau	0.005	Polyak update after critic step.
Actor / critic LR	3e-4 / 3e-4	Independent Adam optimizers.
Temperature	Automatic	Target entropy = -3 for three action dimensions.
Update-to-data ratio	1 initially	Benchmark 1, 2, and 4 after correctness.
Updates start	After warm-up	Do not train critics on an extremely narrow initial distribution.

7.4 SAC update pseudocode

```
while environment_steps < budget:
action = random_action() if replay.size < warmup else actor.sample(obs)
transition = vector_env.step(action)
replay.add(transition_with_final_observation)
obs = reset_finished(transition.next_obs)
if replay.ready():
for update in range(update_to_data_ratio):
batch = replay.sample(batch_size)
update_twin_critics(batch)
update_actor_and_temperature(batch)
polyak_update_target_critics()
periodic_evaluate_and_checkpoint()
```

8. PPO versus SAC in this project

Dimension	PPO	SAC	Project implication
Data use	On-policy; discard after epochs	Off-policy replay	SAC should need fewer simulated transitions, PPO is easier to reason about.
State	Actor + value + rollout	Actor + 2 critics + 2 targets + replay	SAC has greater memory and checkpoint complexity.
Parallel envs	Strongly benefits from many envs	Can work with fewer envs	Existing worker-copy pattern helps both.
Stability	Sensitive to batch/advantage/clip details	Sensitive to critic targets and reward scale	Both need fixed evaluation seeds and component metrics.
Exploration	Policy std + entropy bonus	Maximum-entropy objective + alpha	SAC often explores continuous controls more persistently.
GPU utilization	Large rollout minibatches	Repeated large replay batches	Both can use CUDA; SAC may keep GPU busier.
First milestone	Recommended	After PPO parity	Avoid debugging environment and replay semantics simultaneously.

Selection policy

Treat neither algorithm as the permanent winner. Run both under the same environment-step budget, fixed evaluation suite, and wall-clock reporting. Choose the deployment policy by completion-rate confidence and lap-time distribution, not by training return alone.

9. RLtools and CUDA integration

9.1 Integration boundary

Vendor or fetch a pinned RLtools revision and hide its template-heavy API behind rltools_env_adapter.h and algorithm-specific trainer translation units. Application code should not include RLtools headers. This reduces compile-time spread and protects the simulation API from library-version changes.

Project type	RLtools concept	Adapter responsibility
RacingEnv	Environment specification + state	Expose observation/action dimensions and step/reset operations.
EnvConfig	Environment parameters	Copy immutable track/config references or owned training data safely.
NormalizedAction	Action matrix row	Map [-1, 1]^3 to physical controller targets.
StepResult	Reward + termination	Preserve termination/truncation semantics and final observations.
TrainingMetrics	Loop state / evaluation	Translate RLtools metrics to project-owned telemetry structs.

9.2 Device strategy

Stage	Environment	Networks and optimizer	Data movement
Correctness	Single CPU env	CPU	None; compare with unit tests.
Throughput baseline	128 CPU envs	CPU	Establish simulation ceiling.
Hybrid CUDA	128-256 CPU envs	CUDA	Contiguous observation/action/transition batches per control step or rollout.
Optional GPU env	Only after profiling	CUDA	Requires SoA dynamics and GPU-friendly track queries; high engineering cost.

Use page-locked staging buffers only after ordinary contiguous transfers are measured. Keep observations and actions resident on device across optimizer epochs. For PPO, upload each rollout batch once; for SAC, store replay on host initially and upload sampled batches. A device replay buffer is a later optimization.

9.3 Build design

```
cmake -S . -B build -A x64 ^
-DRACING_ENABLE_CUDA=ON ^
-DRACING_BUILD_VIEWER=ON ^
-DRACING_BUILD_TRAINING=ON
cmake --build build --config Release --target racing_train racing_evaluate racing
```

- Require x64 for CUDA training; keep Win32 configurations unsupported for RL targets.
- Pin RLtools by commit or release and record it in checkpoint metadata and build output.
- Compile CUDA-specific trainer code only when the toolkit is detected; always retain a CPU test path.
- Generate the Visual Studio solution from CMake. The existing racing.vcxproj can remain temporarily but should not be the canonical dependency definition.
- Add a startup compatibility report: GPU name, compute capability, CUDA runtime/driver, precision, and selected backend.

9.4 Precision and determinism

Use float32 for networks and simulation initially. Do not enable mixed precision until loss parity is established. Determinism has tiers: exact CPU unit tests, seeded algorithm reproducibility within one build/backend, and statistical reproducibility across devices. Record seed, git revision, library revision, build type, and hardware for every run.

10. Trainer, storage, and concurrency

10.1 Vectorized environments

Replace the GA candidate array with VectorRacingEnv, a trainer-owned set of independent RacingEnv states. At first use one environment per worker chunk, not one Windows thread-pool callback per policy. Parallelize across environments with stable worker ownership so each state and RNG stays on one worker.

- Immutable track geometry and configuration are shared read-only.
- Mutable car/controller/history/RNG state is owned per environment.
- Observation and action batches are contiguous [environment][feature] arrays.
- Completed environments reset immediately, while their terminal/final observations remain in storage.
- The renderer receives a copied snapshot from a selected evaluation environment; it never reads a state being mutated by workers.

10.2 Storage layouts

Field	PPO rollout	SAC replay
Observation	T x N x 64	Capacity x 64
Action	T x N x 3	Capacity x 3
Reward	T x N	Capacity
Flags	terminated + truncated	terminated + truncated
Next/final observation	Final observation for bootstrap	Explicit next observation or next index + final copy
Algorithm extras	value, old log-prob, advantage, return	none in replay; priorities optional and deferred

10.3 UI and trainer concurrency

Training should run in the headless executable by default. If in-process training is retained, use a dedicated trainer thread and publish immutable TrainingSnapshot records through a single-producer/single-consumer queue. Stop requests are cooperative and checked between collection/update phases. Do not post algorithm-owned pointers through Win32 messages.

11. Checkpoints, configuration, and telemetry

11.1 Checkpoint bundle

Replace RACENN4 with a versioned directory or single archive. Keep the old loader only as an optional GA import tool; a deterministic GA policy cannot be losslessly converted into PPO/SAC critic and optimizer state.

Artifact	Required content
manifest.json	format version, algorithm, observation/action schema, dimensions, architecture, seed, step counters, git/RLtools revision
policy.bin	Actor weights needed for inference
training_state.bin	Critic(s), target critic(s), optimizer moments, PPO log std or SAC temperature
normalization.bin	Observation count, mean, variance, and optional return statistics
config.json	Resolved environment, reward, algorithm, and evaluation configuration
metrics.csv	Append-only training and evaluation metrics

11.2 Metrics

Category	Metrics
Task	return, lap completion, lap time percentiles, progress, off-track, stationary, sector reach
Control	action mean/std, saturation, steering variation, throttle-brake conflict, lateral-demand excess
PPO	policy/value loss, entropy, explained variance, clip fraction, approx KL, gradient norm
SAC	Q1/Q2 loss, Q means, target Q, actor loss, alpha, entropy, replay age, update-to-data ratio
Performance	environment steps/s, inference batches/s, optimizer updates/s, GPU utilization, host-device transfer time

11.3 Viewer changes

- Replace GA generation/population fields with algorithm, environment steps, updates, evaluation return, completion rate, best median lap, and device.
- Add algorithm selector and configuration file picker only if in-app training remains a requirement; the CLI remains the reproducible path.
- Save/Load operates on policy checkpoints and validates observation/action schema before activation.
- Run mode uses deterministic actor output and a fixed 1/60 s simulation tick independent of rendering rate. Rendering interpolates or skips frames without changing dynamics.

12. Validation strategy

12.1 Tests before learning

Test	Method	Pass condition
Reset determinism	Reset same seed/options twice and compare state + observation	Bitwise equal on the same CPU build
Step parity	Replay fixed action trace through old and extracted env	State/reward difference within defined float tolerance
Observation bounds	Fuzz resets/actions over many steps	No NaN/Inf; documented bound violations only
Reward identity	Recompute sum from RewardTerms	Matches total reward within 1e-6 absolute
Wrap progress	Cross start line forward/backward	Small signed delta, no full-lap spike
Termination semantics	Force off-track, stationary, lap, and timeout	Correct exclusive terminated/truncated flags
Action mapping	Use -1, 0, +1 vectors	Exactly reaches documented physical targets
Checkpoint round trip	Save, reload, compare deterministic actions	Same actions and normalization statistics

12.2 Algorithm smoke tests

- Overfit a tiny deterministic track segment or scripted one-dimensional progress task.
- Confirm PPO policy ratio is exactly one before the first update and advantages are approximately zero-mean after normalization.
- Confirm SAC target critics do not receive gradients and both critics learn from identical target values.
- Run a short CPU/GPU parity case with the same frozen batch and compare losses/parameter deltas within tolerance.
- Ensure a random policy, zero action, and current GA checkpoint each produce stable reference metrics before RL training begins.

12.3 Evaluation protocol

Every reported checkpoint is evaluated without exploration over a fixed matrix of at least 10 seeds x 3 legacy starts plus a random-start hold-out set. Report median and 10th/90th percentiles, not only the best lap. Training and evaluation environments must be separate instances; evaluation must not update normalization statistics.

Acceptance area	Initial milestone target
Correctness	All environment and checkpoint tests pass in Debug and Release x64.
Learning	Both PPO and SAC exceed random and zero-action baselines on return and sector reach in at least 4 of 5 seeds.
Completion	At least one algorithm reaches >=80% lap completion on the fixed-start suite.
Robustness	No more than 15 percentage-point completion drop on randomized start hold-out.
Performance	CUDA training update is faster than CPU for the configured batch; total pipeline bottleneck is measured.
Reproducibility	Every run directory contains resolved config, revisions, seed, metrics, and recoverable checkpoint.

13. Migration roadmap

Phase	Deliverables	Exit criteria	Estimate
0 - Baseline	Freeze GA behavior; add reference traces and metrics	Repeatable baseline artifacts	1-2 days
1 - Headless core	Split umbrella header; extract track/env/controller; fixed-step CLI	Old/new step parity tests pass	3-5 days
2 - Vector env	Batched states, reset distribution, contiguous buffers, benchmark	Stable seeded 128-env execution	3-4 days
3 - RLtools CPU	Pinned dependency, environment adapter, tiny smoke environment	PPO and SAC compile/run on CPU	3-5 days
4 - PPO	Actor/critic, rollout, GAE, logging, checkpoint	PPO beats baselines across seeds	5-8 days
5 - SAC	Replay, twin critics, targets, alpha tuning	SAC beats baselines across seeds	5-8 days
6 - CUDA	GPU trainers, batch transfers, profiling	Verified loss parity and speedup	3-6 days
7 - Viewer	Inference loader, RL telemetry, deterministic playback	Viewer loads both policy types	2-4 days
8 - Tuning	Curriculum, reward ablations, algorithm comparison	Acceptance suite satisfied	5-10 days

Do not delete the GA path immediately

Keep the current GA trainer behind RACING_ENABLE_GA_LEGACY through the PPO parity milestone. It is a valuable environment regression oracle and baseline. Remove it only after the RL checkpoint/viewer path and evaluation reports are stable.

13.1 Suggested first pull requests

PR	Scope	Review focus
PR 1	Introduce CMake and headless env test target without changing behavior	Build reproducibility and dependency boundaries
PR 2	Split racing.h and extract fixed-step RacingEnv	Parity, no Windows types in core
PR 3	Add VectorRacingEnv, run config, metrics, benchmark	Ownership, RNG isolation, throughput
PR 4	Pin RLtools and add CPU PPO smoke trainer	Adapter isolation and termination semantics
PR 5	Complete PPO + checkpoints + evaluation	GAE, log probability, metrics, repeatability
PR 6	Add SAC behind same trainer contract	Replay correctness, targets, alpha
PR 7	Enable CUDA and viewer policy loading	Device parity, graceful CPU fallback

14. Risks and mitigations

Risk	Impact	Mitigation
RLtools API/version drift	Build breakage or configuration churn	Pin revision, isolate adapter, add compile smoke target.
GPU does not improve wall time	Complexity without benefit	Measure CPU simulation, transfer, and update time separately; keep CPU backend.
Reward hacking	Fast but invalid behavior	Component telemetry, boundary tests, video/trace review, task metrics independent of return.
Non-Markov observation	Unstable value/Q estimates	Retain history; add explicit lateral/heading error if ablation shows ambiguity.
Action saturation	Oscillation and poor exploration	Log saturation/std, use squashed distributions correctly, tune physical ranges.
Replay memory pressure	SAC allocation or cache issues	Packed records, configurable capacity, host replay first.
UI/trainer races	Crashes or corrupted snapshots	Separate process by default; immutable snapshot exchange if embedded.
Checkpoint incompatibility	Cannot reproduce/deploy runs	Manifest versioning, schema validation, round-trip tests, atomic rename on save.
False algorithm comparison	Wrong engineering decision	Same step budget, seeds, starts, evaluation cadence, and reported wall clock.

15. Implementation checklist

- [ ] Record baseline GA traces, metrics, binary format, and build settings.
- [ ] Create CMake x64 targets for viewer, train, evaluate, benchmark, and tests.
- [ ] Extract Windows-free core, environment, controller, reward, and observation headers.
- [ ] Add fixed control tick and explicit terminated/truncated behavior.
- [ ] Version obs-v1 and action-v1; add running observation normalization.
- [ ] Build deterministic VectorRacingEnv with independent RNG streams.
- [ ] Pin RLtools and implement the project-owned adapter boundary.
- [ ] Implement PPO rollout/GAE/loss/metrics/checkpoint/evaluation.
- [ ] Implement SAC replay/twin critics/targets/alpha/checkpoint/evaluation.
- [ ] Add CUDA backend, device report, CPU fallback, and parity benchmark.
- [ ] Update Direct2D telemetry and inference-only policy runtime.
- [ ] Run multi-seed fixed and hold-out evaluation; publish comparison report.

Appendix A. Proposed configuration skeleton

```
{
"run": {"algorithm": "ppo", "seed": 1, "device": "cuda"},
"environment": {
"track": "data/gb-1948.geojson",
"control_step": 0.016666667,
"physics_max_step": 0.01,
"action_repeat": 1,
"max_episode_steps": 12000,
"observation_schema": "obs-v1",
"action_schema": "high-level-v1"
},
"reward": {
"progress_per_lap": 10.0, "time_per_second": 0.01,
"steering_change": 0.002, "off_track": 2.0,
"stationary": 1.0, "lap_completion": 2.0
},
"ppo": {
"envs": 128, "horizon": 256, "gamma": 0.995,
"gae_lambda": 0.95, "clip": 0.2, "learning_rate": 0.0003,
"epochs": 10, "minibatch": 2048, "target_kl": 0.02
},
"sac": {
"envs": 64, "replay_capacity": 1000000, "warmup": 20000,
"batch": 1024, "gamma": 0.995, "tau": 0.005,
"learning_rate": 0.0003, "auto_temperature": true
},
"evaluation": {"interval_steps": 100000, "fixed_seeds": 10}
}
```

Appendix B. Source-to-target mapping

Current symbol/file	Target location	Change
racing.h: Track, CarParameters, CarState	core/track.h, env/types.h	Move with minimal semantic change.
racing.h: RacingEnv / Action / Observation / StepResult	env/racing_env.h	Add config, schemas, terminated/truncated contract.
simulation.cpp: track geometry	core/track.cpp	Separate parsing/geometry from car integration.
simulation.cpp: observe/step/reset	env/racing_env.cpp	Fixed tick, normalized action adapter, reward terms.
simulation.cpp: vehicle_controller_update	control/vehicle_controller.cpp	Preserve deterministic high-level controller.
NeuralPolicy + genome conversion	legacy/ga_policy.*	Keep only under legacy flag; not part of RL runtime.
training.cpp	legacy/ga_training.cpp + rl/*	Split GA oracle from PPO/SAC trainers.
main.cpp	app/main_win32.cpp	Replace GA commands/telemetry with policy checkpoint and optional trainer client.
rendering.cpp	app/rendering.cpp	Consume project-owned TrainingSnapshot.
racing.vcxproj	CMakeLists.txt	Generate Visual Studio project; retain temporary compatibility.

Appendix C. References

- Schulman, J. et al. Proximal Policy Optimization Algorithms, 2017. https://arxiv.org/abs/1707.06347
- Haarnoja, T. et al. Soft Actor-Critic Algorithms and Applications, 2018. https://arxiv.org/abs/1812.05905
- Eschmann, J., Albani, D., and Loianno, G. RLtools: A Fast, Portable Deep Reinforcement Learning Library for Continuous Control, JMLR 25(301), 2024. https://www.jmlr.org/papers/v25/24-0248.html
- RLtools repository and examples, including PPO and SAC CUDA examples. https://github.com/rl-tools/rl-tools
- RLtools documentation: environment and loop interfaces. https://docs.rl.tools/
- NVIDIA CUDA Toolkit documentation. https://docs.nvidia.com/cuda/

Local references use the workspace snapshot inspected on 04 September 2026; symbols remain the durable references if line numbers move.